MERN STACK

DAY – 39 8 August 2026

PROJECT: PARKEASY - PARKING MANAGEMENT SYSTEM

VEHICLE MANAGEMENT

• The Vehicle Management module is developed to allow administrators to monitor and manage vehicles entering and exiting the parking area.

• It enables administrators to view vehicle details, search vehicles, filter records by date, and manage vehicle entry and exit operations.

• The useState() hook is used to manage vehicle data, search values, selected dates, and filtered vehicle records.

• The useEffect() hook is used to automatically load vehicle details when the page is opened.

• The Vehicle Management module communicates with the backend using REST APIs for retrieving and updating vehicle information.

Vehicle Management APIs

GET /vehicles

• Retrieves all vehicle booking records from the database.

• Fetches owner details, vehicle number, vehicle type, slot details, booking date, timing, and parking status.

• Displays complete vehicle information in the administrator panel.

PUT /vehicle-entry/:userId/:index

• Records vehicle entry when an approved booking vehicle arrives at the parking area.

• Checks whether the booking is approved before allowing entry.

• Validates the booking date and time before updating the parking status.

• Updates entry time and changes parking status to Parked.

PUT /vehicle-exit/:userId/:index

• Records vehicle exit after the vehicle leaves the parking area.

• Calculates the parking duration based on entry and exit time.

• Updates the booking status as Completed.

• Changes parking status from Parked to Exited.

Working

1. The VehicleManagement() component is created to manage vehicle monitoring operations.

2. The component loads all vehicle records using the GET /vehicles API.

3. Vehicle information is displayed in a table format containing owner, vehicle, slot, date, time, status, and parking status.

4. The search feature allows administrators to find vehicles using vehicle numbers.

5. Date filtering allows administrators to view vehicle records for a specific date.

6. The Today button automatically selects the current date and displays today's vehicle records.

7. The Vehicle Entry button sends a request to the PUT /vehicle-entry/:userId/:index API.

8. The backend validates booking approval, date, and time before allowing vehicle entry.

9. The Vehicle Exit button updates the vehicle exit details using the PUT /vehicle-exit/:userId/:index API.

10. After entry or exit operations, vehicle records are refreshed automatically.

11. The Vehicle Management module helps administrators track parking activity and maintain accurate vehicle movement records.

ADMIN PROFILE MANAGEMENT

• The Admin Profile Management module is developed to allow administrators to view and update their account information.

• It enables administrators to edit profile details such as name and email address.

• The profile update process communicates with the backend using REST API.

Admin Profile API

PUT /update-admin/:id

• Updates administrator profile information in the database.

• Modifies administrator name and email details.

• Saves updated profile information permanently in MongoDB.

Working

1. The AdminProfile() component is created to manage administrator profile details.

2. The admin user ID is retrieved from localStorage after login.

3. Existing administrator details are loaded using the GET /user-dashboard/:id API.

4. Profile information is displayed inside a profile card interface.

5. Administrators can edit their name and email fields.

6. The Save Profile button sends updated information to the PUT /update-admin/:id API.

7. The backend verifies the administrator record before updating details.

8. Updated profile information is stored in MongoDB.

9. A success message is displayed after successful profile modification.

10. The Admin Profile Management module provides a simple and secure way for administrators to maintain their account information.

```
app.get("/vehicles",async(req,res)=>{
try{
const users=await User.find();
let vehicles=[];
users.forEach(user=>{
user.bookings.forEach((booking,index)=>{
vehicles.push({
userId:user._id,
bookingIndex:index,
owner:user.name,
email:user.email,
vehicleNumber:
booking.vehicle ||
user.vehicle?.number ||
"Not Added",
vehicleType:
user.vehicle?.type ||
"Unknown",
slot:booking.slot,
date:booking.date,
time:
booking.time ||
`${booking.startTime} - ${booking.endTime}`,
startTime:booking.startTime,
endTime:booking.endTime,
status:booking.status,
entryTime:
booking.entryTime || "",
exitTime:
booking.exitTime || "",
parkingStatus:
booking.parkingStatus || "Not Entered",
duration:
booking.duration || 0
});
});
});
res.json(vehicles);
}
catch(err){
res.status(500).json({
message:err.message
});
}
});
```

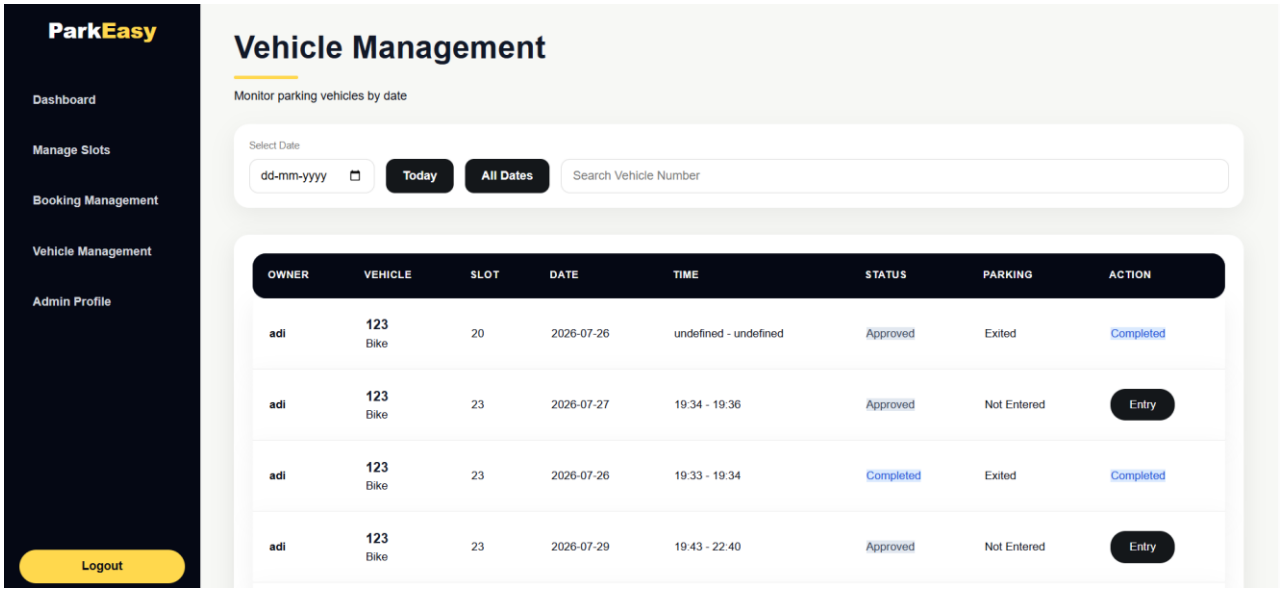
```
app.put("/vehicle-entry/:userId/:index",
async(req,res)=>{
try{
const user=await User.findById(req.params.userId);
if(!user){
return res.json({
success:false,
message:"User not found"
});
}
const booking=user.bookings[Number(req.params.index)];
if(!booking){
return res.json({
success:false,
message:"Booking not found"
});
}
if(booking.status!=="Approved"){
return res.json({
success:false,
message:"Booking is not approved"
});
}
const now=new Date();
const today=
now.toISOString().split("T")[0];
if(today!==booking.date){
return res.json({
success:false,
message:"Vehicle entry allowed only on booking date"
});
}
const currentTime=
now.toTimeString().slice(0,5);
if(
currentTime < booking.startTime ||
currentTime > booking.endTime
){
return res.json({
success:false,
message:
"Vehicle entry not allowed outside booking time"
});
}
booking.entryTime=new Date().toISOString();
booking.parkingStatus="Parked";
await user.save();
res.json({
success:true,
message:"Vehicle entry recorded"
});
}
```

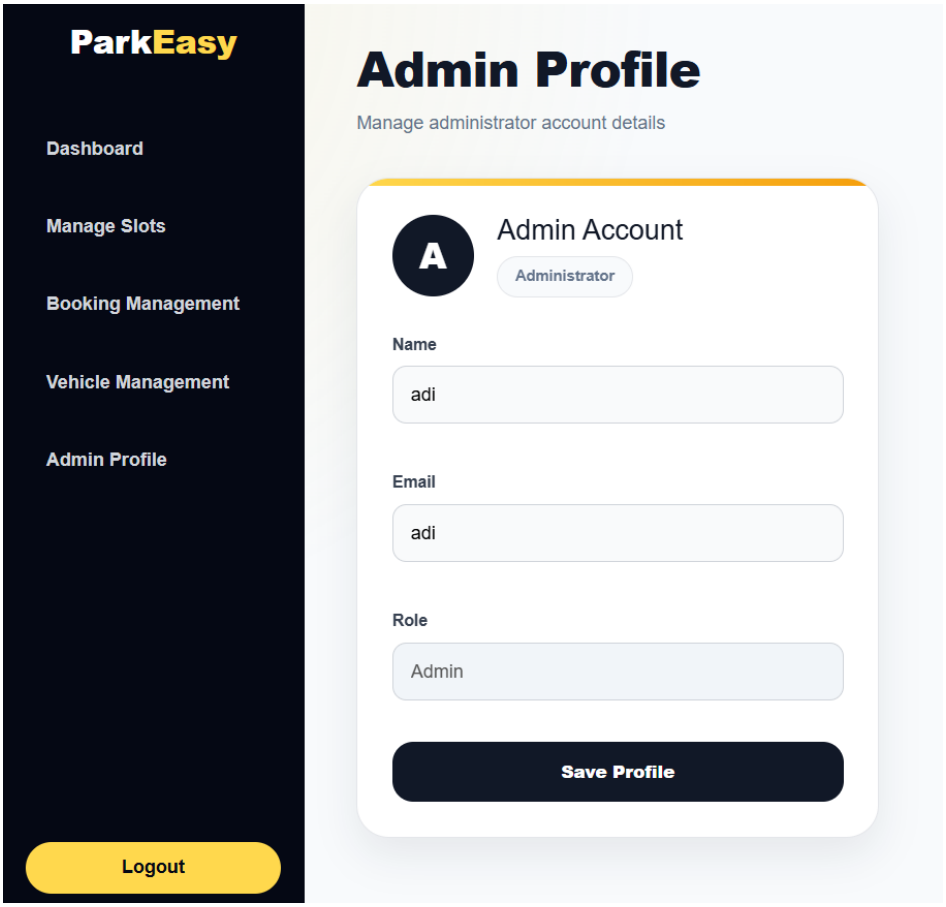
```
catch(err){
res.status(500).json({
success:false,
message:err.message
});
}
});
app.put("/vehicle-exit/:userId/:index",
async(req,res)=>{
try{
const user=await User.findById(req.params.userId);
if(!user){
return res.json({
success:false,
message:"User not found"
});
}
const booking=user.bookings[Number(req.params.index)];
if(!booking){
return res.json({
success:false,
message:"Booking not found"
});
}
if(booking.parkingStatus!=="Parked"){
return res.json({
success:false,
message:"Vehicle is not inside parking"
});
}
const exit=new Date();
const entry=new Date(booking.entryTime);
const hours=Math.ceil(
(exit-entry)/(1000*60*60)
);
booking.exitTime=
exit.toISOString();
booking.duration=
hours>0 ? hours : 1;
booking.parkingStatus="Exited";
booking.status="Completed";
await user.save();
res.json({
success:true,
message:"Vehicle exit completed",
duration:booking.duration
});
}
catch(err){
res.status(500).json({
success:false,
message:err.message
});
}
```

```
app.put("/update-admin/:id",async(req,res)=>{
try{
const admin=await User.findById(req.params.id);
if(!admin){
return res.json({
success:false,
message:"Admin not found"
});
}
admin.name=req.body.name;
admin.email=req.body.email;
await admin.save();
res.json({
success:true,
message:"Admin profile updated successfully"
});
}
catch(err){
res.status(500).json({
success:false,
message:err.message
});

}

});
```





By : Aditi Tangri URN : 2302460 CRN : 2315004